小林 智輝 / Tomoki Kobayashi

GPUの性能を引き出す研究と、生成AIを活用したセキュリティリスク分析・対策検証に取り組んでいます。

北陸先端科学技術大学院大学（JAIST）情報科学系・修士1年。数学科出身。
強みは、課題を論理的に整理し、**分析・実装・検証・改善まで一貫して進める力**です。

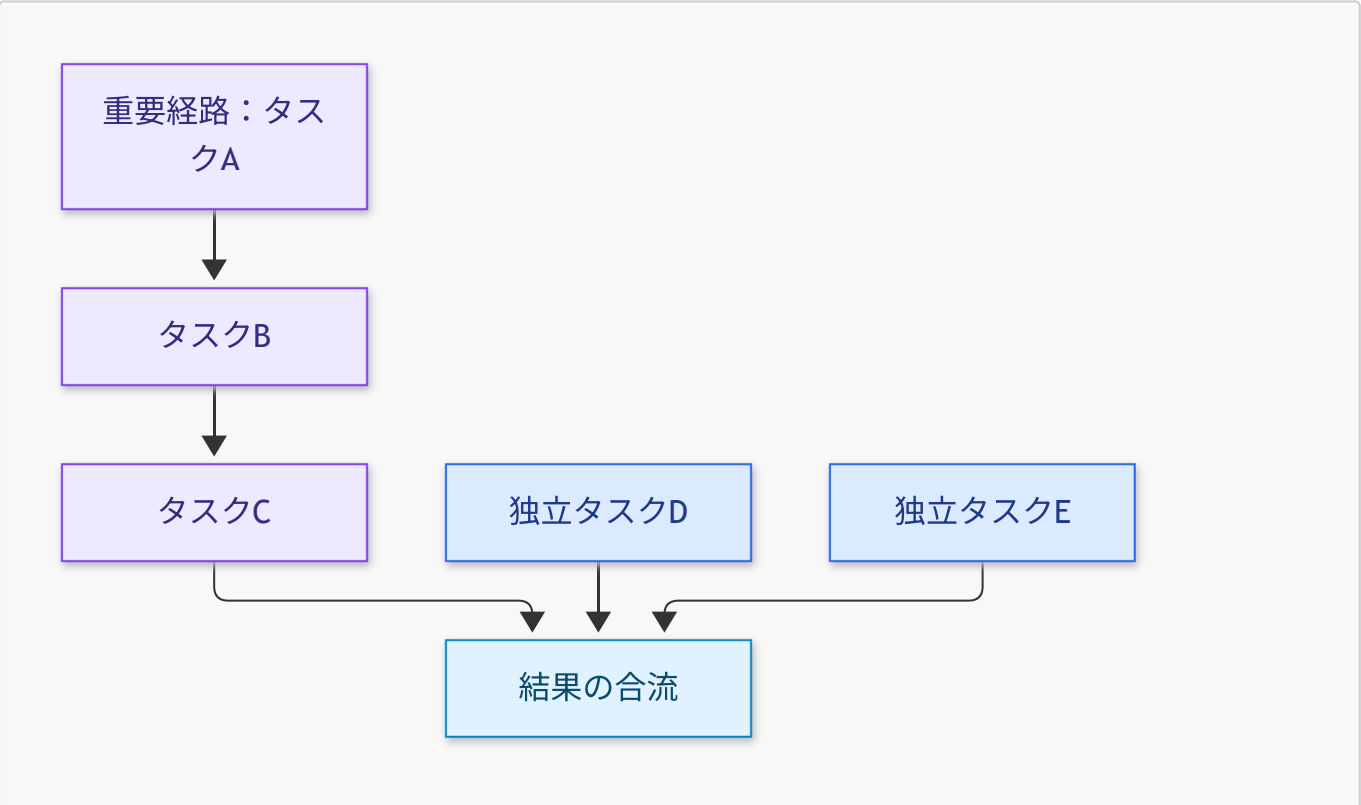
修士研究 · AIセキュリティリスク分析 · セキュリティ基礎学習 · 自主開発

修士研究｜GPUタスクスケジューリング・資源配分

AIなどの計算処理を高速化するため、**タスクの依存関係とGPU計算資源の配分を考慮したスケジューリング手法**を研究しています。

複数の処理を並列に実行しても、重要な処理が資源競合で遅れると、全体の完了が遅くなります。
そこで、処理の重要度に応じて実行先とGPU資源を配分し、全体の実行時間を短縮する方法を検討しています。

対象とする依存構造の概念例



重要な依存経路を進めながら、独立タスクの並列実行を維持することを目指します。

取り組み	内容
分析・設計	依存グラフとbottom levelからタスク重要度を算出
実装	C++・CUDAでStream配置と計算資源（SM）の配分を実装

取り組み	内容
評価	NVIDIA H100上で11ベンチマーク・4手法を比較
改善	実行時間とGPU稼働率からボトルネックを分析

現時点の測定では、長い依存経路と並列タスクが混在する合成グラフで、既存手法比**約9.1%の実行時間短縮**を確認しました。

一方、実アプリケーション由来の6種類では改善に至っておらず、カーネルの処理特性を反映した予測モデルの改善を検討しています。

使用技術： C++ / CUDA / CUDA Stream / CUDA Green Context / Taskflow / Nsight Systems・Compute

→ 研究コード・測定条件・評価結果：master_study

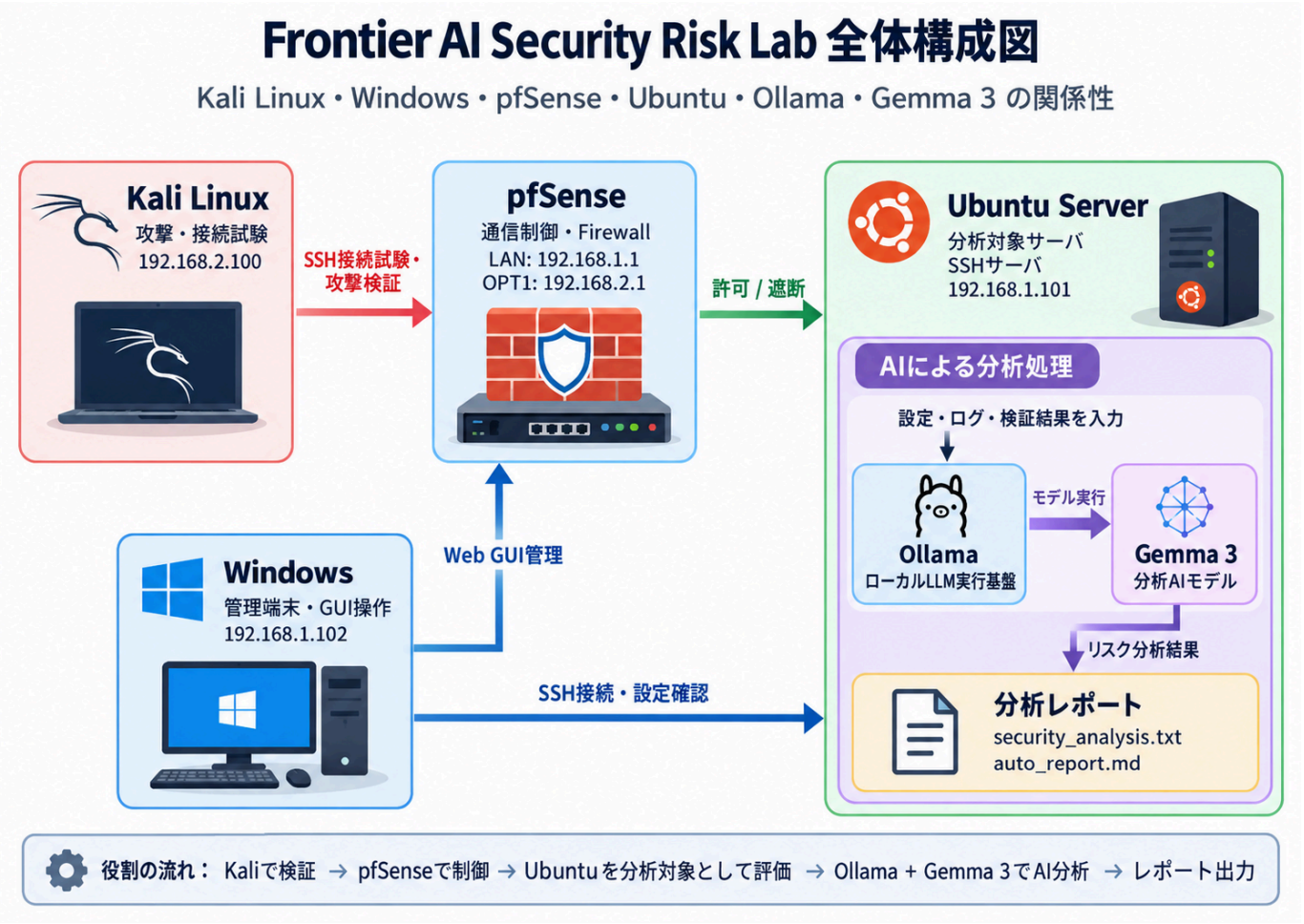
セキュリティ｜生成AIを活用したリスク分析・対策検証

サイバー攻撃による被害を防ぎ、誰もが安心してICTサービスを利用できる環境を支えることを目指して、セキュリティの自主学習に取り組んでいます。

現在は、NTTドコモビジネスのCRXソリューションにおける「フロンティアAI」を参考に、**生成AIによるセキュリティリスク分析から対策・効果検証までの一連の流れを疑似体験**しています。

実際のCRX製品を使用したものではなく、その取り組みを参考に構築した独自の学習環境です。

検証環境の全体構成

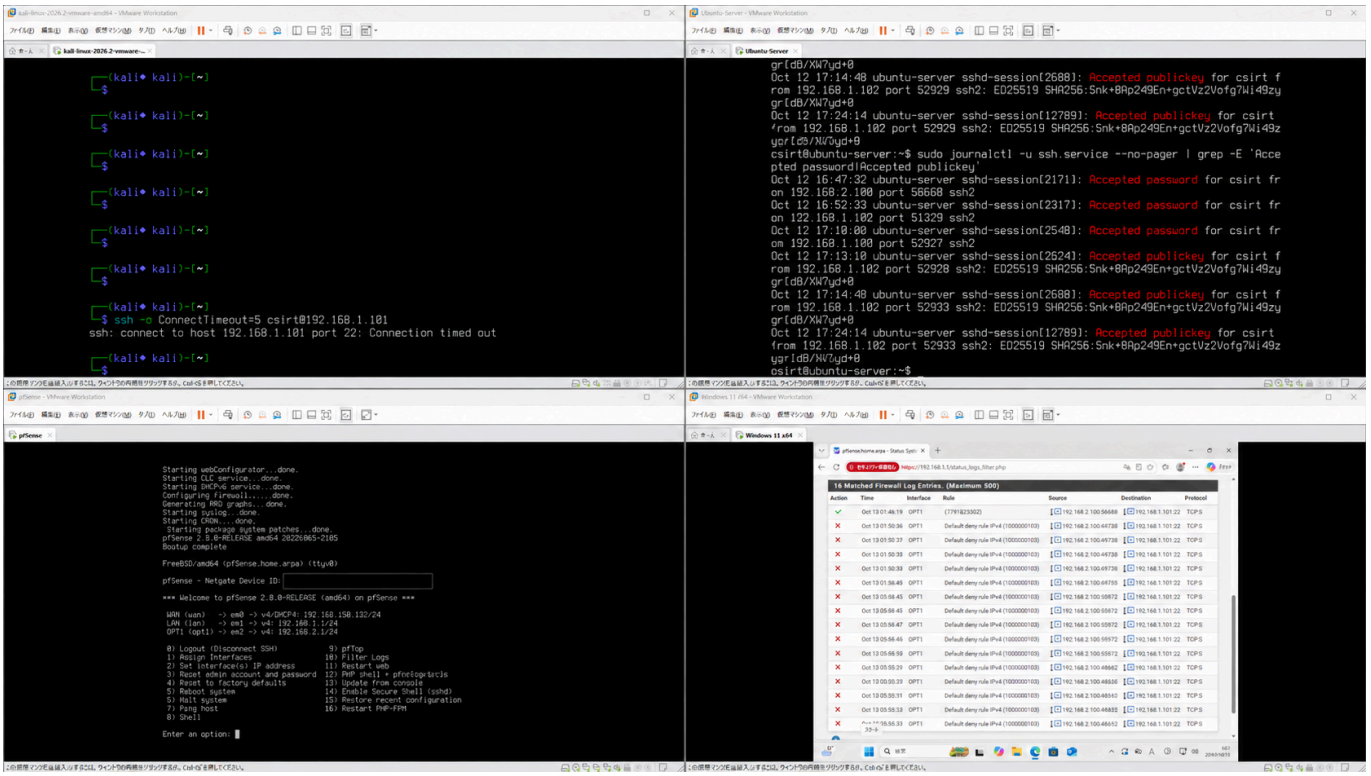


VMware Workstation上にKali Linux、Ubuntu Server、Windows 11、pfSenseを構築し、SSH接続のセキュリティ検証を実施しています。

さらに、Ubuntu Server上でOllama・Gemma 3を活用したローカルAI分析環境を構築しました。

構成要素	役割
Kali Linux	SSH接続試験・セキュリティ検証
Ubuntu Server	分析対象サーバー・SSHサービス
Windows 11	正規の管理端末・pfSenseのGUI管理
pfSense	ネットワーク間の通信制御・ログ記録
Ollama	ローカルLLMの実行基盤
Gemma 3	SSH設定・通信制御に関するリスク分析
Python	AI分析処理・結果保存・レポート生成

検証画面



実際の仮想環境でSSH接続試験とログ確認を行いました。

- **Kali Linux（左上）：** 対策後のSSH接続がタイムアウトすることを確認
- **Ubuntu Server（右上）：** パスワード認証から公開鍵認証への移行をログで確認
- **pfSense（左下）：** WAN・LAN・OPT1のネットワーク構成と通信制御を確認
- **Windows 11（右下）：** 正常なSSH接続を確認

取り組み｜SSH接続設定のリスク分析・対策検証

Ubuntu ServerのSSH設定とネットワーク構成を分析し、不要な接続を遮断しながら正常な管理接続を維持できるか検証しました。

工程	取り組み
情報収集	UbuntuのSSH設定とネットワーク構成を確認
リスク分析	認証方式や接続許可範囲のリスクを分析
実機検証	KaliからUbuntuへのSSH接続試験を実施
対策	pfSenseによる接続元制限とSSH公開鍵認証への移行
効果検証	接続試験とファイアウォール・認証ログから対策結果を確認

検証結果

管理端末以外からもSSH接続できる状態を確認し、不要な通信を遮断しながら正常な管理接続を維持できることを検証しました。

- **対策前：** KaliからUbuntuへのSSH接続に成功
- **対策後：** pfSenseでKaliからのSSH通信を遮断
- **正常接続：** WindowsからUbuntuへのSSH接続を維持

- **認証強化：** パスワード認証を無効化し、公開鍵認証へ移行
- **ログ分析：** pfSenseの遮断ログとUbuntuの認証ログで結果を確認

WindowsとUbuntuは同一LANに配置しており、WindowsからのSSH通信は通常pfSenseを経由しません。

この実験では、**Kaliからの通信を遮断しつつ、正常な管理経路を維持できること**を確認しました。

現在の進捗｜ローカル生成AIによるリスク分析

SSH接続の制御・認証強化に加え、**OllamaとGemma 3（4B）を用いたローカルAIによるセキュリティリスク分析**に取り組んでいます。

Ubuntu Server上にローカルAIの実行環境を構築し、PythonからOllama APIを呼び出すことで、SSH設定・ファイアウォールルール・実際の接続試験結果を分析する仕組みを作成しました。

工程	現在の進捗
仮想環境構築	完了：Kali・Ubuntu・Windows・pfSense
SSH接続制御	完了：pfSenseによる接続元制限
SSH認証強化	完了：公開鍵認証への移行
ログによる効果検証	完了：通信ログ・認証ログを確認
ローカルAI分析	実施：Ollama・Gemma 3で設定リスクを分析
AI分析結果の評価	実施：AIの判断と実際の接続試験結果を比較
分析の自動化	Pythonによる分析・レポート生成スクリプトを作成
Webアプリへの展開	今後の予定

AI分析から得られた知見

生成AIはSSH設定からリスクを抽出できる一方で、**無効化されたファイアウォールルールを誤って解釈するなど、実際の通信状態と異なる判断をする場合がある**ことを確認しました。

そこで、SSH設定・ネットワーク構成・実際の接続試験結果をAIに入力し、判断の根拠を再評価する仕組みを実装しました。

この取り組みでは、**AIの回答をそのまま採用せず、通信試験やログによって検証し、分析結果の信頼性を高めること**を重視しています。

今後の取り組み

今後はAIへの入力・回答と実測結果を一連の実験記録として保存し、分析の再現性を高めます。

さらに、Webアプリケーションの認証・アクセス制御にも検証対象を広げ、リスクの発見から対策・効果検証までの分析力を高めていきます。

使用技術： Python / Ollama / Gemma 3 / VMware Workstation / Kali Linux / Ubuntu Server / Windows 11 / pfSense / OpenSSH

→ 実験構成・検証結果・AI分析コード：Frontier-AI-Security-Risk-Lab

→ セキュリティ基礎学習：security

その他の取り組み

プロジェクト	内容
学士研究	PythonでQ学習・DQNを実装。迷路・CartPoleで収束速度と学習の安定性を比較
エアコン遠隔操作	サーバーとデバイスの連携を実装。擬似デバイスで動作を検証
大学院入試の実装演習	プログラミング問題をCで再現・実装

技術

開発： C++ / CUDA / Python / C / JavaScript

環境・分析： Linux / Git / CMake / VMware Workstation / pfSense / OpenSSH / Ollama / Gemma 3 / Nsight Systems / Nsight Compute

Contact

GitHub：kcs-j02